المدرسة العليا للتكنولوجيا
École Supérieure de Technologie
ⵜⴰⵎⴰⵡⴰⵙⵜ ⵏ ⵓⵙⴳⵎⵉ ⴰ ⵜⵓⵙⵙⵏⵜ ⴰⵣ ⵓⵏⴰⵎ

Chapitre 1
Les bases du langage Python

Programmation Python/M245

Anas MOKHTARI
a.mokhtari@ump.ac.ma

2025/2026

CDL, Département Informatique, Ecole Supérieure de Technologie - UMP, Oujda

Plan du chapitre

Introduction

Syntaxe de base

Structures de contrôle

Structures répétitives

Conclusion

Introduction

Introduction : Caractéristiques

- Langage de programmation interprété : Exécution ligne par ligne
- Objectif : facilité en utilisation
- Peut être utilisé en mode interactif ou par script
- Langage de très haut niveau
- Favorise la programmation impérative structurée et la programmation orientée objet
- Doté d'un typage dynamique fort, d'une gestion automatique de la mémoire et d'un système de gestion d'exceptions
- Langage portable : Windows, Linux, ...
- Open source

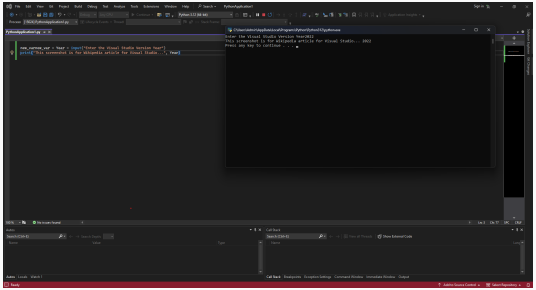
Introduction : Caractéristiques

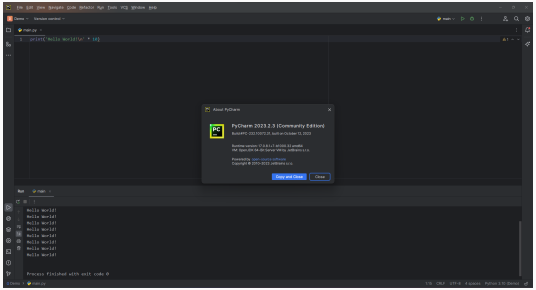
- Langage polyvalent :
  - Web : Django, Flask, ...
  - Calcul scientifique : Numpy, Matplotlib, ...
  - GUI : PyGTK, PyQT, ...
  - Images : Pillow, ...
  - Machine learning : Scikit-Learn, . . .
  - Etc.
- Versions :
  - Première fois publiée en 1991
  - Deux versions majeures (incompatibles) : 2 et 3
  - Dernière version Python 2 : 2.7
  - Version stable actuelle[1] Python 3 : 3.13

1. Février 2026

Introduction : Installation

- Windows & MacOS : `https://www.python.org`
- Linux : installé par défaut
- IDE : MS Visual Studio, PyCharm, IDLE, ...





Syntaxe de base

Syntaxe de base : Littéral et variables

- Littéral :
  - Donnée que sa valeur est déterminée par le littéral même.
  - `123`, `"c"`, `0x123`, `0b1101` et `3e8` (`3x108`) sont des littéraux
  - `True` et `False` sont aussi des littéraux (valeurs booléennes)
  - Le littéral spécial `None` est utilisé pour représenter l'absence de valeur
- Variables :
  - La variable est déclarée en lui affectant une valeur
  - Python est de typage dynamique
  - Noms des variables : Python est sensible à la casse
  - Casting de type :
    - en entier : `int(variable)`
    - en float : `float(variable)`
    - Une valeur numérique en chaîne de caractères : `str(variable)`
  - La fonction `type(variable)` retourne le type de la variable.

Syntaxe de base : Opérateurs arithmétiques

- Arithmétiques : `+, -, *, /, //, %, **`
- Unaires : `-` et `+ (+123)`
- Les opérateurs opèrent de gauche à droite, sauf l'opérateur `**` :
  - L'expression `9 % 6 % 2` donne 1
  - L'expression `2 ** 2 ** 3` donne 256
- Ordre de priorité des opérateurs :
  1. `+, -` (unaire)
  2. `**`
  3. `*, /, //, %`
  4. `+, -` (binaire)
- Les expressions entre parenthèses sont calculées en premier.

Syntaxe de base : Opérateurs arithmétiques

Exemple 1 :

```
a = 3
b = a
c = 10 ** a
d = a * c
print(a, b, c, d)
3    3    1000    3000
```

Exemple 2 :

```
var = 1
print(var)
1
var = "abcd"
print(var)
abcd
```

Exemple 3 :

```
a = 5
b = a
a = b + 3
print(a, b)
8        5
```

Syntaxe de base : Opérateurs de comparaison

- `==, !=, <, <=, >, >=`
- Ordre de priorité des opérateurs :
  1. `+, -` (unaire)
  2. `**`
  3. `*, /, //, %`
  4. `+, -` (binaire)
  5. `<, <=, >, >=`
  6. `==, !=`

Syntaxe de base : Opérateur de comparaison

Exemple :

```
>>> 10 < 100
True
>>> 10 < 9
False
>>> (10 < 100) and (10 < 9)
False
>>> (10 < 100) and not (10 < 9)
True
>>> 10 + True
11
>>> 10 + False
10
>>> 10 + None
Traceback (most recent call last):
File "<stdin>", line 1, in <module>
TypeError: unsupported operand type(s) for +: 'int' and 'NoneType'
```

Syntaxe de base : Opérateurs logiques

- `and`, `or`, `not`
- Ordre de priorité des opérateurs logiques :
  1. `not`
  2. `and`
  3. `or`
- Exemple :

```
i = True
j = not not i #j reçoit True
```

Syntaxe de base : Opérateurs bitwise

- `&, |, ~, ^`
- Utilisable avec les entiers seulement
- Exemple : pour une variable sur 32 bits

```
i = 15
j = ~i #j reçoit -16
```

- Ordre de priorité des opérateurs :
  1. `~, +, -` (unaire)
  2. `**`
  3. `*, /, //, %`
  4. `+, -` (binaire)
  5. `<<, >>`
  6. `<, <=, >, >=`
  7. `==, !=`
  8. `&`
  9. `|`

Syntaxe de base : Opérateurs bitwise

Exemple 1 :

```
>>> 10 * 2 + 3
23
>>> 10 * (2 + 3)
50
>>> 10 ** 2
100
>>> 10 ** -1
0.1
>>> 10 ** .5
3.1622776601683795
```

Exemple 2 :

```
>>> 10 << 1
20
>>> 10 << 4
160
>>> 10 >> 1
5
>>> 10 >> 2
2
>>> 10 >> 4
0
```

Exemple 3 :

```
>>> 10 ^ 8
2
>>> 10 & 8
8
>>> 10 | 8
10
>>> 10 | 4
14
>>> ~10
-11
```

Syntaxe de base : Types de données [2]

- Numériques : `int` et `float` et `complex`
- Autres : `bool, str, list, tuple, range, dict, NoneType`, ...
- Exemples :

```
>>> a = 2
>>> type(a)
<class 'int'>

>>> b = 5.0
>>> type(b)
<class 'float'>

>>> type(a * b)
<class 'float'>
```

```
>>> type(2//5)
<class 'int'>

>>> print(2//5)
0

>>> type(2/5)
<class 'float'>

>>> print(2/5)
0.4
```

```
>>> 1/0
Traceback (most recent call last):
File "<stdin>", line 1, in <module>
ZeroDivisionError: division by zero

>>> 1//0
Traceback (most recent call last):
File "<stdin>", line 1, in <module>
ZeroDivisionError: integer division by zero
```

2. Python 3

Structures de contrôle

Structure conditionnelle : Syntaxe

Syntaxe :

```
if␣condition:
␣␣␣␣instructions
else:
␣␣␣␣instructions
```

Remarques :

- L'indentation est obligatoire !
- Une indentation de 4 espaces est recommandée
- La tabulation est possible, mais pas de mélange entre les deux

Structure conditionnelle : Syntaxe

Fonctionnement :

- Si condition est `True` ou non nulle, on exécute les instructions. Sinon, on passe à `else`
- `else` n'est pas obligatoire

Exemple 1 :

```
if 7 > 2 :
    print("La condition est vraie")
else:
    print("La condition est fausse")
```

Exemple 2 :

```
chaine = input("Note sur 20:")
note = float(chaine)
if note >= 10.0:
    # Ce bloc est exécuté si
    ↪  l'expression (note >= 10.0)
    ↪  est vraie
    print("j'ai la moyenne")
print("Fin du programme")
```

Structure conditionnelle : Syntaxe

Exemple 3 :

```
chaine = input("Note sur 20:")
note = float(chaine)
if note >= 10.0:
    # Ce bloc est exécuté si l'expression (note >= 10.0) est vraie
    print("j'ai la moyenne")
else:
    # Ce bloc est exécuté si l'expression (note >= 10.0) est fausse
    print("C'est en dessous de la moyenne")
print("Fin du programme")
```

Structure conditionnelle : Syntaxe

Exemple 4 :

```
chaine = input("Note sur 20:")
note = float(chaine)
if note >= 20.0 or note < 0.0:
    print("Note invalide !")
else:
    if note >= 10.0:
        print("J'ai la moyenne")
    else:
        print("C'est en dessous de la moyenne")
print("Fin du programme")
```

Structure conditionnelle : Syntaxe

Exemple 5 :

```
chaine = input("Note sur 20:")
note = float(chaine)
if note >= 20.0 or note < 0.0:
    print("Note invalide !")
else:
    if note >= 10.0:
        print("J'ai la moyenne")
        if note == 20.0:
            print("C'est même excellent !")
    else:
        print("C'est en dessous de la moyenne")
        if note == 0:
            print("... lamentable !")
print("Fin du programme")
```

Structure conditionnelle : if-else imbriquées

Syntaxe :

```
if c1:
    if c2:
        instructions
    else:
        instructions
else:
    instructions
```

Structure conditionnelle : elif

Syntaxe :

```
if c1:
    instructions
elif c2:
    instructions
elif c3:
    instructions
else:
    instructions
```

Structure conditionnelle : elif

Exemple :

```
chaine = float(input("Note sur 20:"))
if note == 0.0:
    print("C'est en dessous de la moyenne")
    print("... lamentable !")
elif note == 20.0:
    print("J'ai la moyenne")
    print("C'est même excellent !")
elif note < 10.0 and note > 0.0:    # ou bien elif 0.0 < note < 10.0:
    print("C'est en dessous de la moyenne")
elif note >= 10.0 and note < 20.0:  # ou bien elif 10.0 <= note <= 20.0:
    print("J'ai la moyenne")
else:
    print("Note invalide !")
print("Fin du programme")
```

Structure conditionnelle : elif

Autre syntaxe :

```
if c1: instruction
else: instruction
```

Structures répétitives

Structures répétitives : while

Syntaxe :

```
while condition:
    instructions
```

Exemple 1 :

```
i = 0
while i < 10:
    print(i)
    i = i + 1
```

Structures répétitives : while

Syntaxe :

```
while condition:
    instructions
```

Exemple 2 :

```
compteur = 1
while compteur < 5:
    print(compteur, compteur < 5)
    compteur += 1
print(compteur < 5)
print("Fin de la boucle")
```

Structures répétitives : while

Syntaxe :

```
while condition:
    instructions
```

Exemple 3 :

```
compteur = 1
while compteur < 10:
    print(compteur, '* 8 =', compteur*8)
    compteur += 1
print("Et voilà !")
```

Structures répétitives : for

Syntaxe :

```
for variable_de_contrôle in plage:
    instructions
```

Exemple 1 :

```
for i in 5, 19, -2:
    print(i)
```

Exemple 2 :

```
chaine = 'Bonsoir'
for lettre in chaine:
    print(lettre)
print("Fin de la boucle")
```

Structures répétitives : for

- `range(a)` : créer les valeurs de 0 à $(a - 1)$
- `range(a, b)` : créer les valeurs de a à $(b - 1)$
- `range(a, b, c)` : c est le pas d'incrémentation

Exemple 1 :

```
for i in range(16):
    print(i)
```

Exemple 2 :

```
for compteur in range(1, 11):
    print(compteur, '* 9 =', compteur*9)
print('Et voilà !')
```

Structures répétitives : utiliser break/continue

- `break` : sortie immédiate de la structure répétitive
- `continue` : sortie immédiate de l'itérative actuelle

Exemple :

```
import time
while True:
    print('Heure courante ', time.strftime('%H!%M:%S'))
    quitter = input('Voulez-vous quitter le programme (o/n) ? ')
    if quitter == 'o':
        break
print("A bientôt")
```

Conclusion